U-MV: Regional-scale Acacia tortilis crown mapping from UAV imagery

Technical hand-over and reproduction guide
Software, environment, data, training, evaluation and regional inference

Document version 1.1 · 09 September 2026
Repository: github.com/brakuta/U-MV-Acacia-tortilis-Crown-Mapping
Reference: Gibril et al. (2026), Int. J. Appl. Earth Obs. Geoinf. 148, 105214, doi:10.1016/j.jag.2026.105214

Prepared by Mohamed Barakat A. Gibril
GIS and Remote Sensing Center, Research Institute of Sciences and Engineering, University of Sharjah
Contact: mbgibril@sharjah.ac.ae

Contents

1 Purpose and scope of this document

This document accompanies the hand-over of the U-MV (U-shaped MambaVision) framework for *Acacia tortilis* crown mapping. It is written for a team member who has (i) the public repository and (ii) read access to the project archive `A.tortilis_Data & Model` on the shared drive, which holds the dataset and the original MMSegmentation work directories with the trained checkpoints. It enables that person to install the software, verify it, reproduce the published accuracy figures and produce regional crown maps from new UAV orthomosaics without further assistance.

The document consolidates the repository documentation (`README.md`, `docs/01` to `docs/08`, `docs/REVISION_NOTES.md`) into one reference. The repository remains the authoritative source; where the two differ, the repository is more recent.

1.1 How to use this document

- **Chapter 3 is the shortest path**: what to copy, how to configure, and a step table with an expected outcome for every step. A reader in a hurry can work from chapter 3 alone.
- Chapter 2 describes the framework; chapters 4 to 8 give the full installation, data, training, evaluation and inference reference behind the steps of chapter 3.
- Chapter 9 maps the paper to the configuration files and records the reconciliation with the original training logs. Chapter 10 lists failure modes with remedies.
- Appendices record what changed in the code revision of September 2026, describe the released weights, and provide a command reference.

1.2 Conventions

Commands are given for a shell inside the Docker container, where the dataset is mounted at `/data` and the checkpoint folder at `/weights`. Paths on the host are placeholders and must be adapted. Windows paths seen from WSL2 have the form `/mnt/<drive>/...`; paths containing spaces must be quoted.

2 Framework overview

U-MV couples the hierarchical MambaVision encoder of Hatamizadeh and Kautz (2025), a hybrid of Mamba state-space mixers, self-attention and convolution, with a lightweight U-Net-style decoder. The encoder produces feature maps at strides 4, 8, 16 and 32; the decoder upsamples the deepest map stage by stage, concatenating skip features and applying two 3 × 3 convolution–BatchNorm–ReLU blocks per stage, and a 1 × 1 convolution yields two-class logits. Three encoder sizes are used: tiny (80/160/320/640 channels), small (96/192/384/768) and base (128/256/512/1024). Training minimises cross-entropy plus three times the Dice loss on 1024 × 1024 UAV tiles at 2.5 cm ground sampling distance.

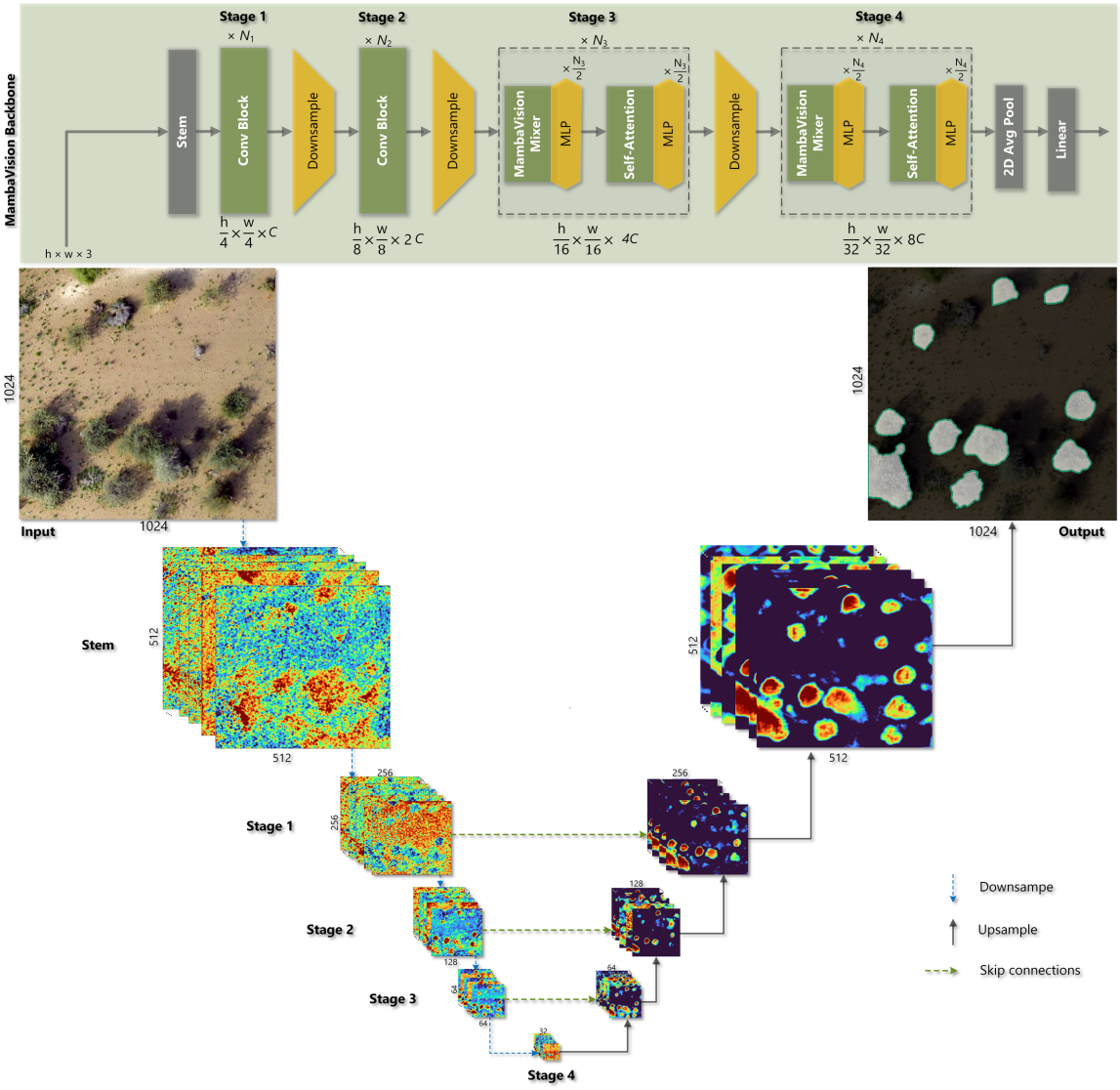

Figure 1. U-MV architecture: four-stage MambaVision encoder (top) and U-Net decoder with skip connections applied to a 1024 × 1024 UAV tile.

2.1 Highlights

- **Three variants** (tiny / small / base; 35.4 M and 54.2 M parameters for tiny and small) with released checkpoints; test-set mIoU 85.30–85.44 % and mF-score 91.52–91.61 %.
- **Self-contained MMSegmentation extension**: `pip install -e .` registers the backbone, decoder and dataset; no files are copied into MMSegmentation.

- **Reproducible environment**: Dockerfile pinned to the original stack (PyTorch 2.6.0 + CUDA 11.8, MMCV 2.1.0, MMSegmentation 1.2.2, mamba-ssm 2.2.4).
- **Regional inference**: seam-free sliding-window prediction over gigapixel orthomosaics, streaming vectorisation to GeoPackage/Shapefile with per-crown `area` and `mean_prob` attributes.

2.2 Repository layout

```
umv/                          installable package (registered with MMSegmentation)
├── models/mamba_vision.py    MambaVisionBackbone (Hugging Face nvidia/MambaVision-*-1K)
├── models/unet_head.py       GenericUNetHead (lightweight U-Net decoder)
├── datasets/uav_acacia.py    UAVAcaciaDataset (background / acacia)
├── inference/                tiling, blending, vectorisation, pipeline, CLI options
└── checkpoint.py             checkpoint inspection and variant detection
configs/
├── _base_/                   models/umv_unet.py · datasets/uav_acacia_dataset.py ·
│                             schedules/schedule_100k_adamw.py · default_runtime.py
└── mambavision/              U-MV-tiny.py · U-MV-small.py · U-MV-base.py
tools/
├── train.py, test.py         MMSegmentation v1.2.2 entry points (+ --test-split)
├── geospatial_inference.py   one orthomosaic -> crown polygons
├── batch_geospatial_inference.py   folder tree -> crown polygons
├── verify_install.py, inspect_checkpoint.py, download_backbones.py, check_dataset.py
docker/                       Dockerfile · docker-compose.yml · run.sh
requirements/                 core.txt · geo.txt · mamba.txt · dev.txt
Pretrained_Weights/           released checkpoints (Git LFS) + README
tests/                        CPU unit tests (tiling, vectorisation, pipeline)
docs/                         01 installation … 08 hand-over checklist, REVISION_NOTES
```

2.3 Published accuracy

Model	Validation mIoU	Validation mF-score	Test mIoU	Test mF-score
U-MV-t	87.91	93.20	85.44	91.61
U-MV-s	88.02	93.27	85.38	91.58
U-MV-b	88.11	93.32	85.30	91.52

Values in percent (paper, Table 1). On the generalisability set (~11 km², 2 165 tiles) U-MV-s reached 89.48 % mIoU and 94.17 % mF-score. Training on a TITAN RTX (24 GB) with batch size 2 took 9.33 h (tiny), 11.8 h (small) and 18.12 h (base) for 100 000 iterations.

3 Hand-over guide: what you receive and how to proceed

This chapter is self-contained. Follow it from top to bottom; every step names a command and the outcome that confirms success. The receiving team member has read access to the project share (`Z:`) where the archive `A.tortilis_Data & Model` resides.

3.1 What you receive

Item	Where	What it is
Code	https://github.com/brakuta/U-MV-Acacia-tortilis-Crown-Mapping	Model, configs, Docker build, tools, tests and this guide (`docs/`).
Dataset	`Z:\...\A.tortilis_Data & Model\Data used to build the model`	`img_dir/` and `ann_dir/` with `train`, `val`, `test2`, `Generalizability` (1024 × 1024 tiles, ~32 GB).
Trained models	`Z:\...\A.tortilis_Data & Model\A.tortilis Models\Pretrained weights`	Three MMSegmentation work directories with the checkpoints, training configs and logs (~2.5 GB).

The archive also contains `A.tortilis Models\MMsegmentation Folder`; that is the superseded container workspace and is **not** needed.

Layout of the two folders to copy:

```
Data used to build the model/                <- DATA_DIR
├── img_dir/{train, val, test2, Generalizability}/*.tif   RGB tiles
└── ann_dir/{train, val, test2, Generalizability}/*.tif   masks: 0 = background, 1 = acacia

Pretrained weights/                          <- WEIGHTS_DIR
├── mambavision-t_generic-unet_acacia/       best_mIoU_iter_100000.pth  (U-MV-tiny)
├── mambavision-s_generic-unet_acacia-88/    best_mIoU_iter_95000.pth   (U-MV-small)
└── mambavision-b_generic-unet_acacia/       best_mIoU_iter_60000.pth   (U-MV-base)
    each with: mambavision-*_generic-unet_acacia.py (training config), iter_100000.pth,
               <timestamp>/ (logs, vis_data/scalars.json)
```

Facts worth knowing before starting:

- **Which weights.** Use `best_mIoU_iter_*.pth` (checkpoint selected on validation mIoU; used for the paper's test results). The files on GitHub (`Pretrained_Weights/U-MV-*_latest.pth`, Git LFS) are byte-identical copies, so `git lfs pull` is unnecessary when the archive is available. Every tool accepts the work-directory folder in place of a file and selects the best checkpoint automatically.
- **Which model.** Start with U-MV-small: it produced the regional maps and has the best generalisability figures. Tiny is fastest; base is largest.
- **Dataset.** Tile counts in the archive are 4 893 (train), 2 407 (val), 3 123 (test2), 2 162 (Generalizability). The published models were trained on 26 615 tiles; the training folder was pruned after training. Evaluation and inference are therefore exact; retraining reproduces the recipe but not the published model.
- **Split names.** The in-distribution test split is `test2`; commands pass `--test-split test2`.
- **Confidentiality.** The dataset is not publicly shareable. Do not redistribute it.

3.2 Prerequisites

- Windows 11 with WSL2 (Ubuntu) and Docker Desktop with the WSL2 backend and GPU support, or a Linux host with Docker and the NVIDIA container toolkit.
- NVIDIA GPU with ≥ 16 GB memory (24 GB used for the published runs) and a driver ≥ 520.
- ~40 GB free on a local SSD for the two folders, ~25 GB for the Docker image.

- Internet access during the image build and the first model build (the MambaVision backbone is downloaded once from the Hugging Face Hub).

3.3 Procedure

Windows paths seen from WSL2 have the form `/mnt/<drive>/...`; paths with spaces must be quoted. Replace `H:` with your local disk.

3.3.1 Copy the two folders to a local disk (PowerShell)

Reading tiles over the network share is too slow for training and evaluation. Copy once, excluding ArcGIS side-car files. Run the commands one at a time (some consoles reorder multi-line pastes):

```
$Z = "Z:\Final Geodatabase\Vegetation_Geodatabase\3_Mapping Acacia tortilis Trees\A.tortilis_Data & Model"
$H = "H:\A.tortilis_Data & Model"
$D = "Data used to build the model"; $W = "A.tortilis Models\Pretrained weights"
robocopy "$Z\$D" "$H\$D" /E /XF *.aux.xml *.ovr *.xml /MT:16 /R:2 /W:5
robocopy "$Z\$W" "$H\$W" /E /MT:16 /R:2 /W:5
```

Expected: in each summary, *Failed* = 0 and *Copied* + *Skipped* = *Total*.

3.3.2 Get the code and point it at the folders (WSL2 terminal)

```
cd ~
git clone https://github.com/brakuta/U-MV-Acacia-tortilis-Crown-Mapping.git
cd U-MV-Acacia-tortilis-Crown-Mapping
cp docker/.env.example docker/.env
nano docker/.env
```

Set the two lines (keep the quotes), save with Ctrl+O, Enter, exit with Ctrl+X:

```
DATA_DIR="/mnt/h/A.tortilis_Data & Model/Data used to build the model"
WEIGHTS_DIR="/mnt/h/A.tortilis_Data & Model/A.tortilis Models/Pretrained weights"
```

3.3.3 Build and start the container

```
# 1. GPU visible to Docker?
docker run --rm --gpus all nvidia/cuda:11.8.0-base-ubuntu22.04 nvidia-smi
# 2. build the image (30-60 min), keeping a log
docker compose --env-file docker/.env -f docker/docker-compose.yml build 2>&1 | tee ~/umv_build.log
# 3. start an interactive container
docker compose --env-file docker/.env -f docker/docker-compose.yml run --rm umv
```

The build compiles MMCV and mamba-ssm (30–60 min). Expected after `run:` a prompt in `/workspace/U-MV`; `ls /data` shows `ann_dir img_dir`; `ls /weights` shows the three `mambavision-*` folders. All remaining commands run inside the container.

3.3.4 Verify, validate, evaluate, map

#	Step	Command	Expected outcome
1	Software stack	`python tools/verify_install.py --variant small`	`RESULT: OK`; forward pass `(1, 3, 512, 512) -> (1, 2, 512, 512)`
2	Cache backbones for offline use	`python tools/download_backbones.py`	three cache paths printed; later `export HF_HUB_OFFLINE=1`
3	Unit tests	`python -m pytest tests -q`	all passed
4	Dataset	`python tools/check_dataset.py /data --splits train val test2 Generalizability`	`RESULT: OK`; pairs 4 893 / 2 407 / 3 123 / 2 162; mask values `[0, 1]`
5	Checkpoint	`python tools/inspect_checkpoint.py "/weights/mambavision-s_generic-unet_acacia-88"`	resolves `best_mIoU_iter_95000.pth`; `variant: small`; `iteration: 95000`

#	Step	Command	Expected outcome
6	Test split	`python tools/test.py configs/mambavision/U-MV-small.py "/weights/mambavision-s_generic-unet_acacia-88" --test-split test2`	mIoU ≈ 85.4 %, mFscore ≈ 91.6 % (paper: 85.38 / 91.58)
7	Generalisability split	same with `--test-split Generalizability`	mIoU ≈ 89.5 %, mFscore ≈ 94.2 % (paper: 89.48 / 94.17)
8	Other variants	repeat 5–7 with `U-MV-tiny.py` + `mambavision-t_generic-unet_acacia`, `U-MV-base.py` + `mambavision-b_generic-unet_acacia`	tiny 85.44 / 91.61; base 85.30 / 91.52 on `test2`
9	One orthomosaic	see command below	`.gpkg` with `area`, `mean_prob`; opens in ArcGIS Pro / QGIS in the orthomosaic CRS
10	Many orthomosaics	`python tools/batch_geospatial_inference.py ... --skip-existing` (see the inference chapter and Appendix C)	mirrored folder of `.gpkg` files + `batch_summary.json`

Step 9 in full. Place the orthomosaic (GeoTIFF, RGB) under the dataset folder so that it is visible at `/data`, e.g. `.../Data used to build the model/orthos/<name>.tif`, then:

```
CKPT="/weights/mambavision-s_generic-unet_acacia-88"
python tools/geospatial_inference.py \
  --config configs/mambavision/U-MV-small.py --checkpoint "$CKPT" \
  --input "/data/orthos/<name>.tif" \
  --output "/data/predictions/<name>_crowns.gpkg" \
  --scratch-dir /tmp/geospatial_work --min-area 1.0 --save-prob
```

Each evaluation run takes 10–20 min on a 24 GB GPU; one 1 km² orthomosaic at 2.5 cm takes a few minutes.

3.3.5 Optional: retrain

```
python tools/train.py configs/mambavision/U-MV-small.py
```

Outputs go to `work_dirs/U-MV-small/` (bind-mounted, persists after exit). Because the archived training split is a subset, expect validation mIoU below the published 88.0 %.

3.4 If something fails

Consult the troubleshooting chapter first. When reporting a problem, include the exact command, the last 50 lines of output, and the result of `python tools/verify_install.py`.

Contact: Mohamed Barakat A. Gibril (mbgibril@sharjah.ac.ae).

4 Software environment and installation

Two routes are supported. The Docker route reproduces the environment in which the published experiments were executed and is the recommended option for hand-over to new team members. The manual route documents every dependency for users who cannot run containers.

4.1 Software stack and version rationale

Component	Version	Reason
Python	3.11	Version of the original container (`docs/reference/environment.frozen.yml`).
PyTorch / torchvision	2.6.0 + cu118 / 0.21.0	Original experiments; CUDA 11.8 wheels run on driver ≥ 520 (TITAN RTX, RTX A5000).
MMEngine	0.10.7	Handles `torch.load(weights_only=True)` introduced in PyTorch 2.6.
MMCV	2.1.0 (full build)	Latest release accepted by MMSegmentation 1.2.2 (`mmcv>=2.0.0rc4,<2.2.0`). No prebuilt wheel exists for PyTorch 2.6, so it is compiled from source.
MMSegmentation	1.2.2	Last stable 1.x release; equivalent to the `main` branch used originally.
transformers / timm	4.50.0 / 1.0.15	Versions with which the MambaVision remote code (`trust_remote_code=True`) is known to import (`timm.models.registry`, `timm.models.layers` shims).
mamba-ssm	2.2.4	Provides `selective_scan_fn`, the CUDA kernel used by the MambaVision mixer. Requires `nvcc`.
GDAL / rasterio / geopandas	3.10 / 1.4 / 1.0 (conda-forge)	GDAL `Polygonize` is used for streaming vectorisation of gigapixel probability rasters.

MambaVision does **not** call `causal_conv1d`; that package is optional.

4.2 Docker route (recommended)

4.2.1 Prerequisites on Windows 11 / WSL2

1 Install an NVIDIA driver ≥ 520 on Windows (the driver is shared with WSL2; do not install a driver inside WSL2).

2 Install Docker Desktop with the WSL2 backend and enable the Ubuntu distribution under *Settings → Resources → WSL integration*.

3 Verify GPU visibility from WSL2:

```
docker run --rm --gpus all nvidia/cuda:11.8.0-base-ubuntu22.04 nvidia-smi
```

4 Grant WSL2 enough memory for the 1024 × 1024 training tiles (in `%USERPROFILE%\.wslconfig`):

```
[wsl2]
memory=48GB
swap=16GB
```

then `wsl --shutdown` from PowerShell.

4.2.2 Build

```
git clone https://github.com/brakuta/U-MV-Acacia-tortilis-Crown-Mapping.git
cd U-MV-Acacia-tortilis-Crown-Mapping
git lfs install && git lfs pull            # released checkpoints; skip if the project archive is available
cp docker/.env.example docker/.env && nano docker/.env   # DATA_DIR, WEIGHTS_DIR
docker compose --env-file docker/.env -f docker/docker-compose.yml build
```

The build compiles MMCV and mamba-ssm and typically takes 30–60 min. The `TORCH_CUDA_ARCH_LIST` variable in `docker/Dockerfile` covers Turing (TITAN RTX, 7.5), Ampere (RTX A5000, 8.6; A100, 8.0), Ada (8.9) and Hopper (9.0); extend it for other GPUs before building.

4.2.3 Run

```
cp docker/.env.example docker/.env        # set DATA_DIR and WEIGHTS_DIR (quoted if they contain spaces)
docker compose --env-file docker/.env -f docker/docker-compose.yml run --rm umv
# inside the container
python tools/verify_install.py --variant small
```

The compose file bind-mounts the repository at `/workspace/U-MV`, the dataset at `/data`, the checkpoint folder read-only at `/weights`, a named volume for the Hugging Face cache and `work_dirs/` for training outputs. `docker/run.sh` offers the same without compose and reads the same `docker/.env`.

4.2.4 Offline use

The MambaVision code and ImageNet weights are downloaded once from the Hugging Face Hub into the `hf_cache` volume:

```
python tools/download_backbones.py --variants tiny small base
export HF_HUB_OFFLINE=1      # subsequent runs need no network
```

4.3 Manual route (conda)

Install in this order; later steps compile against earlier ones.

```
conda create -n umv python=3.11 -y && conda activate umv
# 1. PyTorch (CUDA 11.8 wheels)
pip install torch==2.6.0 torchvision==0.21.0 --index-url https://download.pytorch.org/whl/cu118
# 2. MMEngine and MMCV (compiled; requires nvcc from CUDA 11.8 toolkit on PATH)
pip install mmengine==0.10.7
MMCV_WITH_OPS=1 FORCE_CUDA=1 TORCH_CUDA_ARCH_LIST="7.5;8.6" pip install --no-build-isolation mmcv==2.1.0
# 3. Mamba kernels
pip install packaging ninja && pip install --no-build-isolation -r requirements/mamba.txt
# 4. Geospatial stack (GDAL bindings from conda-forge)
conda install -c conda-forge gdal=3.10 rasterio=1.4 geopandas=1.0 shapely=2.1 pyproj=3.7 pyogrio -y
# 5. Project
pip install -r requirements/core.txt
pip install --no-deps -e .
python tools/verify_install.py
```

`pip install -e .` makes `umv` importable from any working directory; the scripts in `tools/` also work without installation because they add the repository root to `sys.path`.

4.4 Verification

```
python tools/verify_install.py                    # imports, CUDA, mamba_ssm, mmcv.ops
python tools/verify_install.py --variant small    # builds U-MV-small, forward pass, peak VRAM
python -m pytest tests -q                         # tiling / vectorisation unit tests (CPU)
```

Expected: `RESULT: OK`, and for the second command a forward pass of a 512 × 512 tensor producing logits of shape `(1, 2, 512, 512)`.

5 Data

5.1 Source data

The framework was developed on RGB orthomosaics acquired with a senseFly eBee X (S.O.D.A. camera) at ~122 m AGL, processed in Pix4Dmapper to a ground sampling distance (GSD) of 2.5–3 cm. Annotations were produced with the semi-automated workflow described in Section 3.1 of the paper (SAM-LoRA crown proposals, Mask2Former–Swin candidate detection, visual verification).

5.2 Tiles and masks

Orthomosaics and rasterised crown polygons were partitioned into 1024 × 1024 pixel image–mask pairs. This tile size was chosen so that a mean crown (~17.5 m², ~5 m diameter, ~28 000 pixels at 2.5 cm) is embedded in sufficient context (shadows, terrain transitions, neighbouring land cover).

Requirements for new data:

Item	Specification
Images	3-band RGB, 8-bit, `.tif` (GeoTIFF or plain TIFF). Other suffixes: set `img_suffix` in the dataset config.
Masks	Single-band 8-bit index raster, `0 = background`, `1 = acacia`, `.tif` (or `.png` with `seg_map_suffix='.png'`). Not RGB, not 0/255.
Names	Identical basenames for image and mask within a split.
Size	1024 × 1024 recommended; other sizes work (the pipeline resizes to 1024 on the long side and random-crops 1024 × 1024 during training).
Ignore value	255 in a mask is ignored by the loss (`seg_pad_val=255`).

Directory layout (mounted at `/data` in the container):

```
/data
├── img_dir/
│   ├── train/             4 893 pairs in the archive (26 615 used for the published models)
│   ├── val/               2 407
│   ├── test2/             3 123   (in-distribution test)
│   └── Generalizability/  2 162   (out-of-distribution test, separate region)
└── ann_dir/
    ├── train/  val/  test2/  Generalizability/
```

The archived training folder was pruned after the published runs (see the reproducibility chapter); the other three splits are complete.

A different root is selected without editing configs:

```
python tools/train.py configs/mambavision/U-MV-small.py \
  --cfg-options train_dataloader.dataset.data_root=/data/UAV_Acacia \
                val_dataloader.dataset.data_root=/data/UAV_Acacia \
                test_dataloader.dataset.data_root=/data/UAV_Acacia
```

5.3 Integrity check

```
python tools/check_dataset.py /data --splits train val test2 Generalizability
```

reports pair counts, tile sizes, dtypes, mask values and ArcGIS side-car files (`*.tif.aux.xml`, `*.tif.ovr`) per split. Side-cars are created when tiles are opened in ArcGIS Pro; the loader ignores them (only files ending in `.tif` are listed), but they can be deleted from working copies to save space.

A split whose folder is not named `test` (for example `test2`) is evaluated without renaming: `python tools/test.py <config> <ckpt> --test-split test2`.

5.3.1 Manual check

```
import numpy as np, rasterio, glob, os
root = '/data'
for split in ['train', 'val', 'test2', 'Generalizability']:
    imgs = sorted(glob.glob(f'{root}/img_dir/{split}/*.tif'))
    anns = sorted(glob.glob(f'{root}/ann_dir/{split}/*.tif'))
    assert [os.path.basename(p) for p in imgs] == [os.path.basename(p) for p in anns], split
    with rasterio.open(imgs[0]) as im, rasterio.open(anns[0]) as an:
        assert im.count == 3 and im.dtypes[0] == 'uint8', im.profile
        assert an.count == 1 and set(np.unique(an.read(1))) <= {0, 1, 255}, an.profile
        assert (im.width, im.height) == (an.width, an.height)
    print(split, len(imgs), 'pairs OK')
```

5.4 Export from ArcGIS Pro (summary)

1 Rasterise the verified crown polygons to the orthomosaic grid (`Polygon to Raster`, cell size = orthomosaic cell size, value field = 1, NoData → 0).

2 Export image and mask tiles with the same fishnet (`Split Raster`, tile size 1024 × 1024, format TIFF, no overlap for training tiles).

3 Convert masks to 8-bit unsigned (`Copy Raster`, pixel type 8_BIT_UNSIGNED) and verify with the script above.

5.5 Location of the dataset

The dataset folder can reside anywhere; its path is supplied once in `docker/.env` (`DATA_DIR`) or on the command line (`--cfg-options train_dataloader.dataset.data_root=...`). Prefer a local SSD over a network share, because training performs ~30 000 random tile reads per epoch. When copying, ArcGIS side-cars can be excluded:

```
robocopy "<source>\Data used to build the model" "D:\UAV_Acacia_Data" /E /XF *.aux.xml *.ovr *.xml /MT:16
```

If a network drive must be read directly from WSL2, mount it first (`sudo mkdir -p /mnt/z && sudo mount -t drvfs Z: /mnt/z`) and quote the path.

5.6 Orthomosaics for regional inference

Inference operates on whole orthomosaics. Convert them to Cloud-Optimised GeoTIFF (COG) once; windowed reads are then efficient from any storage:

```
gdal_translate -of COG -co COMPRESS=DEFLATE -co BLOCKSIZE=512 -co NUM_THREADS=ALL_CPUS in.tif out_cog.tif
```

6 Training

6.1 Model and optimisation settings

All settings are declared in `configs/` and inherited by the three variant files; nothing is hard-coded in Python.

Setting	Value	Config location
Encoder	MambaVision-T/S/B, ImageNet-1K initialisation (Hugging Face Hub)	`_base_/models/umv_unet.py` → `model.backbone`
Encoder widths	T (80, 160, 320, 640) · S (96, 192, 384, 768) · B (128, 256, 512, 1024)	`mambavision/U-MV-*.py`
Decoder	U-Net style, widths (256, 128, 64, 32), BN + ReLU, bilinear ×2 upsampling	`model.decode_head`
Loss	CE + 3 × Dice (Eq. 1 of the paper)	`decode_head.loss_decode`
Optimiser	AdamW, lr 1e-4, β = (0.9, 0.999), weight decay 0.05; backbone lr multiplier 0.1 (effective 1e-5); no decay on normalisation layers	`_base_/schedules/schedule_100k_adamw.py`
Schedule	100 000 iterations, polynomial decay (power 0.9), validation every 5 000 iterations	same
Gradient clipping	L2 norm 0.01	`optim_wrapper.clip_grad`
Batch	2 × 1024 × 1024 tiles	`_base_/datasets/uav_acacia_dataset.py`
Augmentation	RandomResize (0.5–2.0), RandomCrop 1024 (`cat_max_ratio=0.75`), horizontal flip, photometric distortion	same
Checkpointing	every 5 000 iterations, plus `best_mIoU_iter_*.pth` on validation mIoU	`default_hooks.checkpoint`
Test-time inference	sliding window 1024 × 1024, stride 512	`model.test_cfg`

All three variants share this schedule; the training logs of the released checkpoints confirm it (`docs/07_reproducibility.md`, §7.2). Note that the archived training split is a pruned subset (4 893 of the 26 615 tiles used for the published models), so retraining from the hand-over folder will not reach the published accuracy exactly.

6.2 Commands

```
# inside the container, dataset mounted at /data
python tools/train.py configs/mambavision/U-MV-tiny.py
python tools/train.py configs/mambavision/U-MV-small.py
python tools/train.py configs/mambavision/U-MV-base.py
```

Useful options (inherited from MMSegmentation):

Option	Effect
`--work-dir work_dirs/umv_small_run2`	Output directory (default `work_dirs/<config-name>`).
`--resume`	Continue from the latest checkpoint in the work directory.
`--amp`	Automatic mixed precision (halves activation memory; not used for the published runs).
`--cfg-options train_dataloader.num_workers=16`	Override any config key.
`--cfg-options randomness.seed=0 randomness.deterministic=True`	Fixed seed and deterministic cuDNN.

Outputs in the work directory: `<timestamp>/<timestamp>.log`, `vis_data/scalars.json` (loss and metric curves), `iter_*.pth`, `best_mIoU_iter_*.pth`, and a copy of the resolved configuration.

6.3 Expected resources (paper, TITAN RTX 24 GB, batch 2)

Variant	Parameters	FLOPs (1024²)	Training time (100k it.)	Validation mIoU
U-MV-t	35.41 M	0.144 T	9.33 h	87.91 %
U-MV-s	54.24 M	0.215 T	11.8 h	88.02 %
U-MV-b	not reported	0.396 T	18.12 h	88.11 %

6.4 Practical notes

- **First run downloads the backbone.** The MambaVision code and weights are fetched from the Hugging Face Hub (`nvidia/MambaVision-*-1K`) when the model is built. Pre-download with `tools/download_backbones.py` for offline hosts.
- **DataLoader workers.** The base config uses 8 workers. The original runs used 42 on a 64 GB workstation; increase `train_dataloader.num_workers` if the GPU is starved (check the `data_time` field in the log).
- **WSL2 memory.** Large Mamba backbones can trigger CUDA system-memory fallback and freeze WSL2 when host RAM is exhausted. Mitigations: disable the sysmem fallback in the NVIDIA Control Panel (*CUDA – Sysmem Fallback Policy → Prefer No Sysmem Fallback*), keep `.wslconfig` memory below physical RAM, and drop the page cache between runs (`sync; echo 3 > /proc/sys/vm/drop_caches` from the WSL2 shell as root).
- **Monitoring.** `tail -f work_dirs/<name>/<timestamp>/<timestamp>.log`, or add `vis_backends=[dict(type='LocalVisBackend'), dict(type='TensorboardVisBackend')]` to `configs/_base_/default_runtime.py` (requires `pip install tensorboard`).

7 Evaluation

7.1 Metrics

`IoUMetric` with `iou_metrics=['mIoU', 'mFscore']` reports, per class and averaged over background and acacia, the intersection-over-union, F-score, precision and recall (Equations 2–7 of the paper). Averages are over the two classes (`mIoU`, `mFscore`); the acacia-class values (`IoU.acacia`, `Fscore.acacia`) are the quantities of ecological interest.

7.2 Commands

```
CKPT="/weights/mambavision-s_generic-unet_acacia-88"   # folder -> its best_mIoU_iter_*.pth
python tools/test.py configs/mambavision/U-MV-small.py "$CKPT" --test-split test2              # test
python tools/test.py configs/mambavision/U-MV-small.py "$CKPT" --test-split Generalizability   # OOD
python tools/test.py configs/mambavision/U-MV-small.py work_dirs/U-MV-small --test-split test2 # own run
# released checkpoints (identical to the best_mIoU files; after git lfs pull)
for v in tiny small base; do
  python tools/test.py configs/mambavision/U-MV-$v.py Pretrained_Weights/U-MV-${v}_latest.pth --test-split test2
done
```

Without `--test-split` the config's default split `img_dir/test` is used (the archive names it `test2`).

Options:

Option	Effect
`--test-split NAME`	Evaluate on `img_dir/NAME` + `ann_dir/NAME` (added in this repository).
`--work-dir DIR`	Where the JSON metrics and log are written.
`--out DIR`	Save predicted index masks for offline analysis.
`--show-dir DIR`	Save colour overlays (input, ground truth, prediction).
`--tta`	Multi-scale (0.5–1.75) + horizontal-flip test-time augmentation (slower; not used in the paper).

Evaluation uses the config's sliding-window mode (1024 × 1024 window, stride 512).

7.3 Reference results (paper, Table 1)

Model	Validation mIoU	Validation mF-score	Test mIoU	Test mF-score
U-MV-t	87.91	93.20	85.44	91.61
U-MV-s	88.02	93.27	85.38	91.58
U-MV-b	88.11	93.32	85.30	91.52

Validation covers ~6.5 km² (2 407 archived tiles) and the independent test set ~6.7 km² (3 123 tiles, split `test2`). On the generalisability set (~11 km², 2 162 tiles) U-MV-s reached 89.48 % mIoU and 94.17 % mF-score (§4.4 of the paper). A freshly evaluated released checkpoint should reproduce the published values within about ±0.1 percentage points; larger deviations indicate a data or preprocessing mismatch (band order, mask encoding, image suffix) rather than model drift.

To confirm that a checkpoint matches a config, `python tools/inspect_checkpoint.py <file or work dir>` prints the variant inferred from the decoder shapes together with the iteration and the MMEngine version stored in the checkpoint.

8 Regional inference on orthomosaics

`tools/geospatial_inference.py` (one image) and `tools/batch_geospatial_inference.py` (folder tree) share the implementation in `umv/inference/`. They turn a georeferenced orthomosaic into a GeoPackage (or Shapefile) of crown polygons with per-polygon attributes, without ever holding the full raster in memory.

8.1 Processing steps

1. **Staging (optional, `--scratch-dir`).** The input is copied to a fast local directory. This is recommended on WSL2/Docker when the data reside on a Windows drive (`/mnt/<drive>`), where random window reads are slow.
2. **Tiling.** An edge-aligned grid of `--tile-size` windows with `--overlap` pixels is generated; the last tile of each row/column ends exactly at the raster border, so every tile has full context.
3. **Prediction.** Tiles are normalised as in training (ImageNet mean/std, RGB) and passed through `EncoderDecoder.encode_decode`, yielding logits at tile resolution; the softmax probability of the acacia class is kept.
4. **Blending.** `--blend center` (default) writes only the region between the mid-points of neighbouring overlaps, so every pixel is predicted exactly once from the tile in which it lies farthest from the border (seam-free by construction). `--blend hann` averages all overlapping predictions with a 2-D Hann weight.
5. **Probability raster.** Accumulators are finalised into a tiled, compressed GeoTIFF in the CRS of the input (`--save-prob` keeps it; `--prob-dtype uint8` stores 0–255).
6. **Vectorisation.** The raster is thresholded at `--thresh`, polygonised with GDAL `Polygonize` (streaming, 4- or 8-connectivity), filtered by `--min-area` (CRS units, m² for projected CRS) and enriched with `area` and `mean_prob` (mean probability inside each polygon, for threshold tuning in GIS).

8.2 Commands

```
python tools/geospatial_inference.py \
  --config configs/mambavision/U-MV-small.py \
  --checkpoint Pretrained_Weights/U-MV-small_latest.pth \
  --input  /data/orthos/Fujairah_block12_cog.tif \
  --output /data/predictions/Fujairah_block12_crowns.gpkg \
  --scratch-dir /tmp/geospatial_work --min-area 1.0 --save-prob

python tools/batch_geospatial_inference.py \
  --config configs/mambavision/U-MV-small.py \
  --checkpoint Pretrained_Weights/U-MV-small_latest.pth \
  --input-dir /data/orthos --output-dir /data/predictions \
  --scratch-dir /tmp/geospatial_work --skip-existing --continue-on-error
```

The batch script mirrors the input folder structure, skips finished outputs (`--skip-existing`, resumable) and writes `batch_summary.json`.

8.3 Parameters

Parameter	Default	Guidance
`--tile-size`	1024	Training tile size; keep.
`--overlap`	256	≥ 128. Larger overlap = more context at the write boundary, more compute.
`--blend`	center	`hann` is marginally smoother on ambiguous crowns at a ~30 % higher cost.
`--batch-size`	8	16 fits on 24 GB GPUs in fp16 for U-MV-s; reduce on OOM.
`--precision`	fp16	CUDA autocast; `fp32` for reference comparisons.

Parameter	Default	Guidance
`--thresh`	0.35	Probability threshold used for the regional maps. Tune per region with `mean_prob`; 0.5 is the argmax-equivalent.
`--min-area`	0	e.g. `1.0` m² removes spurious specks; the mean crown is ~17.5 m².
`--connectivity`	4	8 merges diagonally touching pixels.
`--bands`	1 2 3	Band indices fed as R, G, B (for 4-band RGBA orthomosaics `1 2 3` is still correct).
`--band-order`	rgb	**Do not** set `bgr` for standard orthomosaics; see below.
`--prob-dtype`	uint8	`float32` for full-precision probabilities (4× larger).
`--num-workers`	4	Raster reading processes.
`--mp-context`	(torch default)	`spawn` if forked workers misbehave with GDAL.

8.4 Channel order

During training, `LoadImageFromFile` reads TIFFs with OpenCV (BGR order) and `SegDataPreProcessor(bgr_to_rgb=True)` converts them to RGB before normalisation; the network therefore expects **RGB**. Rasterio returns bands in file order, which is RGB for standard orthomosaics, so the inference pipeline applies no channel swap. The previous versions of the scripts swapped the channels whenever `bgr_to_rgb=True` was set, which fed BGR tiles to the network; `--band-order bgr` reproduces that behaviour only for comparison.

8.5 Outputs

File	Content
`<name>.gpkg` (layer `<name>`) or `<name>.shp`	Crown polygons; attributes `area` (CRS units), `mean_prob` (0–1).
`<name>_prob.tif` (with `--save-prob`)	Single-band probability raster; 0–255 for `uint8`.
`batch_summary.json` (batch)	Per-image tiles, polygons, timings, errors.

Post-processing in GIS: select polygons by `mean_prob`, dissolve touching crowns if instance separation is not required, and intersect with survey-zone boundaries.

8.6 Throughput

On a TITAN RTX with U-MV-s, fp16 and batch 8, throughput is roughly 4–6 tiles of 1024 × 1024 per second (≈ 25–40 ha min^{-1} at 2.5 cm GSD), dominated by raster I/O when reading from a Windows-mounted drive; staging to `/tmp` removes that bottleneck.

9 Reproducibility and paper-to-code mapping

9.1 Mapping of Section 3 of the paper to the configuration

Paper statement	Implementation
MambaVision T/S/B encoder, ImageNet-1K initialisation via timm/Hugging Face (§3.3, §3.5)	`MambaVisionBackbone(variant=..., pretrained=True)` → `AutoModel.from_pretrained('nvidia/MambaVision-{T,S,B}-1K', trust_remote_code=True)`
Encoder widths (80,160,320,640) / (96,192,384,768) / (128,256,512,1024) (§3.3)	`decode_head.encoder_channels` in `configs/mambavision/U-MV-*.py`
Lightweight CNN decoder with skip connections (§3.3)	`GenericUNetHead`, widths (256,128,64,32)
L = L_CE + 3 L_Dice (Eq. 1)	`loss_decode=[CrossEntropyLoss(1.0), DiceLoss(3.0)]`
AdamW, β=(0.9,0.999), weight decay 0.05 (§3.5)	`optimizer` in `_base_/schedules/schedule_100k_adamw.py`
Polynomial decay, power 0.9 (§3.5)	`PolyLR(power=0.9, eta_min=0)`
100 000 iterations, validation every 5 000 (§3.5, §4.2)	`train_cfg`, `val_interval=5000`
Batch size 2, tiles 1024 × 1024 (§3.2, §3.5)	`train_dataloader.batch_size=2`, `crop_size=(1024,1024)`
Gradient clipping (§3.5)	`clip_grad=dict(max_norm=0.01, norm_type=2)`
Random crop, horizontal flip, photometric perturbation (§3.5)	`train_pipeline`
Best validation checkpoint retained (§3.5)	`CheckpointHook(save_best='mIoU')`
Metrics: precision, recall, mIoU, mF-score (§3.6)	`IoUMetric(iou_metrics=['mIoU','mFscore'])`
Docker on a TITAN RTX workstation, 64 GB RAM (§3.5)	`docker/Dockerfile` (PyTorch 2.6.0 + CUDA 11.8 base); the original container definition is archived as `docs/reference/Dockerfile.original`

9.2 Reconciliation with the original training logs

The MMSegmentation work directories of the three published runs were recovered in September 2026 and their logs compared with the paper and the released configurations.

1 **Learning rate.** All three runs used a base learning rate of 1×10^{-4} with a backbone multiplier of 0.1; the logs list every encoder parameter at 1×10^{-5}. The paper's "initial learning rate of 1×10^{-5}" therefore refers to the encoder rate. The configs in this repository reproduce the logged setting.

2 **Schedule of the tiny variant.** The tiny log shows 100 000 iterations with validation every 5 000, identical to small and base, and reports the best validation mIoU (87.91 %, as in Table 1 of the paper) at iteration 100 000. The tiny config released earlier (lr 1×10^{-5}, 150 000 iterations) did not correspond to the run and has been aligned with the log.

3 **Checkpoints.** The released files are byte-identical (SHA-256) to `best_mIoU_iter_100000.pth` (tiny), `best_mIoU_iter_95000.pth` (small) and `best_mIoU_iter_60000.pth` (base) of the work directories, i.e. the best-validation checkpoints used for the paper's test results.

4 **Seeds.** No seed was fixed; runs are not bit-reproducible. For controlled comparisons add `--cfg-options randomness.seed=0 randomness.deterministic=True`.

5 **MMSegmentation revision.** The original container installed the `main` branch (`docs/reference/Dockerfile.original`); v1.2.2 is the last tagged release and is API-identical for the components used here.

6 **Channel order at inference.** The original inference scripts swapped channels to BGR before the network; the revised pipeline feeds RGB, which is what the training preprocessor produced

(`docs/05_inference.md`, §5.4). Regional maps produced with the old scripts may therefore differ slightly; `tools/test.py` is unaffected and provides the reference.

7 **Training split of the archived dataset.** The archived `train` folder holds 4 893 tiles (1024 × 1024, identical in every copy found), whereas the paper reports 26 615. The tile indices run from 0 to 19 765 with gaps, and some files carry modification dates after the training runs (August 2025), which indicates that the training folder was pruned after the models were trained. Validation (2 407; the logs evaluate 1 204 batches of 2), test (3 123) and generalisability (2 162) tiles are complete. Consequently, evaluation and inference with the released checkpoints are fully reproducible from the archive, whereas retraining from the archive reproduces the recipe but not the published model.

9.3 What is and is not fixed by the pinned environment

Fixed: library versions, CUDA kernels, preprocessing, architecture, optimiser and schedule. Not fixed: cuDNN algorithm selection (`cudnn_benchmark=True`), data-loading order without a seed, and the exact Hugging Face revision of the MambaVision weights (pin a `revision=` in `MambaVisionBackbone` via `model_name` if the Hub repository is updated upstream).

9.4 Archival

The code is archived on Zenodo (DOI 10.5281/zenodo.18068379). When re-archiving this revision, include `docs/reference/environment.frozen.yml`, the Docker image digest (`docker images --digests umv`), and the SHA-256 of the checkpoints listed in `Pretrained_Weights/README.md`.

10 Troubleshooting

Symptom	Cause	Remedy
`ModuleNotFoundError: No module named 'mmcv._ext'`	`mmcv-lite` or a wheel built for another PyTorch/CUDA is installed.	Reinstall MMCV from source against the active PyTorch (`docs/01_installation.md`, step 2) or use the Docker image.
`No module named 'mamba_ssm'` / `selective_scan_cuda` import error	Kernels not built or built for a different PyTorch.	`pip install --no-build-isolation mamba-ssm==2.2.4` with `nvcc` available; confirm with `tools/verify_install.py`.
`KeyError: 'MambaVisionBackbone is not in the mmseg::model registry'`	`umv` not imported.	Configs must contain `custom_imports = dict(imports=['umv'])` (all shipped configs do); run scripts from the repository root or `pip install -e .`.
`OSError: We couldn't connect to 'https://huggingface.co'`	Backbone download blocked (proxy, offline).	Pre-download on a connected machine (`tools/download_backbones.py`), copy `hf_cache/` and set `HF_HUB_OFFLINE=1`.
`UnpicklingError: invalid load key, 'v'` when loading a `.pth`	The file is a Git LFS pointer.	`git lfs install && git lfs pull` (see `Pretrained_Weights/README.md`).
`_pickle.UnpicklingError: Weights only load failed`	PyTorch ≥ 2.6 with an old MMEngine.	Use MMEngine 0.10.7 (pinned).
`size mismatch for decode_head.decoder_stages.0.0.weight`	Checkpoint and config variant differ.	`python tools/inspect_checkpoint.py <ckpt>` prints the matching config.
Validation mIoU ≈ 50 % or all-background predictions	Masks are 0/255 or RGB, or wrong suffix.	Re-encode masks as 0/1 uint8 single band (`docs/02_data_preparation.md`, §2.3).
Test mIoU several points below the paper	Band order or resolution mismatch.	Ensure RGB orthomosaics, 2.5–3 cm GSD, no `--band-order bgr`.
CUDA out of memory during training	1024² tiles with batch 2 need ~20 GB on U-MV-b.	Use `--amp`, or `train_dataloader.batch_size=1` with doubled iterations.
WSL2 freezes; GPU memory shows 24 GB + system RAM use	CUDA sysmem fallback exhausting host RAM.	Set *Prefer No Sysmem Fallback* in the NVIDIA Control Panel; reduce batch size; drop page cache between runs.
`RuntimeError: DataLoader worker ... Bus error` / shared-memory errors in Docker	`/dev/shm` too small.	Run with `--ipc=host` (compose file does) or `--shm-size=16g`.
`osgeo` missing in inference	GDAL Python bindings absent.	Install via conda-forge; the rasterio fallback only handles rasters ≤ 1.5 gigapixels.
Straight cut lines in the crown map	`--overlap` below ~128 or `--blend` misconfigured.	Keep the defaults (overlap 256, centre-crop).
Very slow inference from `/mnt/<drive>`	Windows filesystem access from WSL2.	Use `--scratch-dir /tmp/geospatial_work` and COG inputs.
`img_ratios` `NameError` when loading a config	Legacy dataset config.	Fixed in this revision; `img_ratios` is defined in `configs/_base_/datasets/uav_acacia_dataset.py`.

A Revision notes (September 2026)

This revision prepares the repository for hand-over and independent reproduction. The network architecture, released configurations and checkpoints are unchanged; the changes concern packaging, correctness of the auxiliary code, portability and documentation.

A.1 Defects found in the previous revision

#	Defect	Effect	Resolution
1	`mmseg/dataset/ade.py` (singular `dataset`) was meant to overwrite MMSegmentation's `mmseg/datasets/ade.py`; the copy-in instructions never placed it there.	Configs referenced `ADE20KDataset`, which expects `.jpg` images and 150 classes; a fresh installation could not load `.tif` tiles or report two-class metrics.	New registered dataset `UAVAcaciaDataset` in `umv/datasets/`; no upstream file is modified.
2	`img_ratios` undefined in the dataset config.	`Config.fromfile` raised `NameError` on every config.	Defined.
3	`configs/_base_/default_runtime.py` absent from the repository.	Configs unloadable outside a full MMSegmentation checkout.	Added (verbatim from v1.2.2).
4	Inference scripts swapped RGB→BGR whenever `bgr_to_rgb=True`.	Network received BGR tiles although it was trained on RGB.	Removed; `--band-order bgr` retained for comparison.
5	Batch script used `mode='whole'` with `ori_shape=(h, w)` for border tiles.	Padded border tiles were resized to the valid size, misaligning predictions along raster edges.	Both scripts now use `encode_decode` at tile resolution and crop.
6	Centre-crop writer double-wrote border regions when a raster edge fell inside the first tile of a row.	Silent averaging at borders.	Edge-aligned grid with mid-point write boundaries; property-tested (`tests/test_tiling.py`).
7	Minimum-area filtering documented in the README but not implemented.	—	`--min-area` implemented; `area` attribute added.
8	Hard-coded paths, thresholds and device settings inside the scripts.	Edits required for every run.	argparse CLI with documented defaults.
9	`num_workers=42` in the dataset config.	Failure or thrashing on smaller hosts.	8 (documented).
10	Backbone always downloaded ImageNet weights, even when loading a fine-tuned checkpoint; no offline path.	Network access mandatory at inference time.	`pretrained=False` when a checkpoint is given; `HF_HUB_OFFLINE` / `local_files_only` supported; `tools/download_backbones.py`.
11	`environment.yml` was a full conda dump of the container's base environment (`prefix: /opt/conda`, 300 packages).	Not recreatable; misleading as a specification.	Moved to `docs/reference/environment.frozen.yml`; replaced by `requirements/*.txt`, `pyproject.toml` and `docker/Dockerfile`.
12	`Assets/mamba_vision_architecture.png` was 92 MB (14 639 × 14 198 px).	Slow clones, README rendering issues.	Downscaled to 2 400 px (2.5 MB); the original remains in Git history.
13	`.idea/workspace.xml` committed.	IDE noise.	Removed and ignored.
14	`tqdm==4.65.2`, `torch==2.6.0+cu118` pins in `requirements.txt` not installable from PyPI without extra index.	`pip install -r requirements.txt` failed.	Split requirements; PyTorch/MMCV/mamba-ssm installed explicitly.

A.2 Structural changes

- `umv/` installable package (`pip install -e .`) with `models/`, `datasets/`, `inference/`, `checkpoint.py`. Registered names `GenericUNetHead`, `mamba_{tiny,small,base}_vision_timm` are kept, and `MambaVisionBackbone` is added, so configs embedded in old checkpoints still resolve.
- Module attribute names (`backbone.backbone.*`, `decode_head.decoder_stages.*`, `decode_head.segmentation_head.*`, `decode_head.conv_seg.*`) are preserved; released checkpoints load without key remapping.
- Configs split into `_base_/models`, `_base_/datasets`, `_base_/schedules`, `_base_/default_runtime.py`; variant files contain only variant-specific fields. Numerical hyper-parameters are unchanged.
- `tools/train.py` and `tools/test.py` vendored from MMSegmentation v1.2.2 (plus `--test-split`), so the repository is self-contained.
- New tools: `verify_install.py`, `inspect_checkpoint.py`, `download_backbones.py`, `batch_geospatial_inference.py` (renamed from `Batch_processing_geospatial_inference.py`).
- `tests/` (14 tests) covering tiling, blending, vectorisation, checkpoint variant detection and the end-to-end inference pipeline on synthetic GeoTIFFs (CPU).

A.3 Compatibility with the archived work directories

`umv/compat.py::load_config` detects configs that import `mmseg.custom_models.*` (the archived training configs) and maps them to the new package, including `ADE20KDataset` -> `UAVAcaciaDataset`. All tools use it, so `tools/test.py <archived config> <archived checkpoint>` works without edits. `docker/.env` (`DATA_DIR`, `WEIGHTS_DIR`) decouples the container from the location of the hand-over folder.

A.4 Validation performed in this revision

- All three configs load; models build with a stub encoder and run a training step (CE + Dice losses), `encode_decode`, and sliding-window prediction on CPU (MMEngine 0.10.7, MMSegmentation 1.2.2).
- The inference pipeline recovers synthetic crowns with correct CRS, areas (±5 %), attributes and seam-free tiling for both blending modes.
- The Git LFS objects of the three released checkpoints are present on GitHub and their SHA-256 values equal those of `best_mIoU_iter_{100000,95000,60000}.pth` in the tiny/small/base work directories.
- The original container definition was recovered and archived as `docs/reference/Dockerfile.original`; it uses the same base image and MMCV build route as `docker/Dockerfile`.

A.5 Not validated here (requires a GPU host)

- The Docker image build (MMCV and mamba-ssm compilation) and the Hugging Face backbone instantiation; the Hub was unreachable from the sandbox in which this revision was prepared. Run `docs/08_handover_checklist.md` steps 5–10 on the workstation and report any deviation.
- Numerical equivalence of the released checkpoints with the paper's metrics.

A.6 Resolved from the recovered work directories (September 2026)

- The tiny configuration has been aligned with its training log (lr 1e-4, 100 000 iterations); the previously released tiny config did not match the run.
- The released checkpoints are the best-validation checkpoints (SHA-256 verified against `best_mIoU_iter_{100000,95000,60000}.pth`).

- The archived training split (4 893 tiles) is a pruned subset of the 26 615 tiles used for the published models; no fuller copy exists on the project disks. Evaluation and inference are exact; retraining reproduces the recipe.

A.7 Decisions left to the authors

1 Whether to publish the checkpoints on Zenodo/Hugging Face in addition to Git LFS (bandwidth quota of GitHub LFS is 1 GB $month^{-1}$ on free plans; three full downloads exhaust it).

2 Whether regional maps produced with the earlier BGR-swapped scripts should be regenerated.

B Pretrained weights

The three released checkpoints are stored with **Git LFS**. A plain `git clone` without LFS yields 134-byte pointer files, not weights. Fetch the binaries with:

```
git lfs install
git lfs pull                          # all three (~815 MB)
git lfs pull --include="Pretrained_Weights/U-MV-small_latest.pth"   # one variant
```

Variant	File	Size	SHA-256 (LFS oid)
U-MV-tiny	`U-MV-tiny_latest.pth`	159 MB	`85796b37582647a6d83973e240ebf30e0a562676bd44ae84eb81c953dc8900ac`
U-MV-small	`U-MV-small_latest.pth`	234 MB	`0c8d2b3dcdbf6272ea12d9fcdac2cba79ca185277cb2f5d1eff55e5d065ed9d8`
U-MV-base	`U-MV-base_latest.pth`	422 MB	`fcd86b17fb1c49e4f9555538526cc0edff805d0303463bbcc9336b7e537b66d3`

Verify a download with `sha256sum <file>` and identify the variant of any local checkpoint with:

```
python tools/inspect_checkpoint.py /path/to/checkpoint.pth
```

A checkpoint obtained elsewhere (e.g. a `best_mIoU_iter_*.pth` from a `work_dirs/` folder) can be used directly by passing its path to `tools/test.py` or the inference scripts; no renaming is required.

B.1 Provenance (verified September 2026)

The released files are byte-identical to the best-validation checkpoints of the original MMSegmentation work directories (SHA-256 verified):

Released file	Work directory	Checkpoint	Iteration
`U-MV-tiny_latest.pth`	`mambavision-t_generic-unet_acacia`	`best_mIoU_iter_100000.pth`	100 000
`U-MV-small_latest.pth`	`mambavision-s_generic-unet_acacia-88`	`best_mIoU_iter_95000.pth`	95 000
`U-MV-base_latest.pth`	`mambavision-b_generic-unet_acacia`	`best_mIoU_iter_60000.pth`	60 000

The `_latest` suffix therefore denotes the latest release, not the last training iteration. `iter_100000.pth` files in the work directories are the final-iteration checkpoints and were not released.

B.2 Original work directories

Passing a work-directory folder to any tool selects its `best_mIoU_iter_*.pth`:

```
python tools/inspect_checkpoint.py "/weights/mambavision-s_generic-unet_acacia-88"
python tools/test.py configs/mambavision/U-MV-small.py "/weights/mambavision-s_generic-unet_acacia-88" --test-split test2
```

C Command reference

C.1 Environment

```
cp docker/.env.example docker/.env                # set DATA_DIR and WEIGHTS_DIR
docker compose --env-file docker/.env -f docker/docker-compose.yml build
docker compose --env-file docker/.env -f docker/docker-compose.yml run --rm umv
python tools/verify_install.py --variant small     # stack check + forward pass
python tools/download_backbones.py                 # cache backbones; then export HF_HUB_OFFLINE=1
python -m pytest tests -q                          # unit tests (CPU)
```

C.2 Data and checkpoints

```
python tools/check_dataset.py /data --splits train val test2 Generalizability
python tools/inspect_checkpoint.py "/weights/mambavision-s_generic-unet_acacia-88"
```

C.3 Training

```
python tools/train.py configs/mambavision/U-MV-small.py
python tools/train.py configs/mambavision/U-MV-small.py --resume --work-dir work_dirs/U-MV-small
python tools/train.py configs/mambavision/U-MV-small.py \
  --cfg-options randomness.seed=0 randomness.deterministic=True
```

C.4 Evaluation

```
CKPT="/weights/mambavision-s_generic-unet_acacia-88"      # folder resolves to best_mIoU_iter_*.pth
python tools/test.py configs/mambavision/U-MV-small.py "$CKPT" --test-split test2
python tools/test.py configs/mambavision/U-MV-small.py "$CKPT" --test-split Generalizability
python tools/test.py configs/mambavision/U-MV-small.py "$CKPT" --test-split test2 \
  --out preds/ --show-dir vis/
```

C.5 Regional inference

```
CKPT="/weights/mambavision-s_generic-unet_acacia-88"
python tools/geospatial_inference.py \
  --config configs/mambavision/U-MV-small.py --checkpoint "$CKPT" \
  --input /data/orthos/block12_cog.tif \
  --output /data/predictions/block12_crowns.gpkg \
  --scratch-dir /tmp/geospatial_work --min-area 1.0 --save-prob

python tools/batch_geospatial_inference.py \
  --config configs/mambavision/U-MV-small.py --checkpoint "$CKPT" \
  --input-dir /data/orthos --output-dir /data/predictions \
  --scratch-dir /tmp/geospatial_work --skip-existing --continue-on-error
```

C.6 Orthomosaic preparation

```
gdal_translate -of COG -co COMPRESS=DEFLATE -co BLOCKSIZE=512 \
  -co NUM_THREADS=ALL_CPUS in.tif out_cog.tif
```